Case Study — Asset Eligibility

Context

OverIT develops a Field Service Management (FSM) platform used to plan, assign, and manage field service interventions and industrial assets.

Before an intervention, the system needs to identify assets (e.g., IoT sensors, machinery, gateways, smart meters) that satisfy a specific set of requirements or health conditions. For example, a sensor may need to operate within a specific temperature range, have an updated firmware version, or be deployed in a designated operational zone.

Different types of operations impose different asset criteria, and these requirements may evolve as new sensor models and telemetry attributes are introduced.

Your task is to design a reusable component that evaluates whether an **asset** satisfies a given set of criteria. The component should be independent of the scheduling and dispatching process itself: it determines whether an asset matches the criteria, not which asset should ultimately be selected or processed.

Your task

Design and implement a Java API for defining and evaluating **asset eligibility criteria**.

Due to the diverse nature of assets, an asset is represented as a `Map<String, String>`, mapping each property or telemetry attribute name to its string value.

The API must allow clients to build filters programmatically, combine them into more complex expressions, and evaluate whether an asset matches the resulting criteria.

The focus of the exercise is on **modelling clarity, extensibility, correctness, and maintainability** rather than framework usage or algorithmic complexity.

Functional requirements

The API should support the following capabilities:

- Evaluate whether an asset matches a filter.
- Support filters that always match or never match an asset.
- Check whether a property/attribute exists in an asset.
- Compare an asset property with a specified value using equality.
- Compare the values of two properties of the same asset (e.g. `currentTemperature` vs `maxAllowedTemperature`).
- Support numeric comparisons, such as greater than, greater than or equal to, less than, and less than or equal to (e.g., evaluating telemetry data or battery levels).

- Combine filters using logical operators such as `AND`, `OR`, and `NOT`.
- Allow filters to be nested and composed programmatically to express more complex operational criteria.
- Provide a meaningful string representation of a filter.

Property names are **case-sensitive**. String equality comparisons must be **case-insensitive**.

You do not need to implement every comparison operator. A representative subset is sufficient, provided the design allows additional operators to be introduced without extensive changes to the existing implementation.

Full parsing and serialization support are out of scope.

Example

Consider the following asset (e.g., a smart telemetry sensor):

```java
Map<String, String> sensor = new HashMap<>();

sensor.put("zone", "Lombardy-Plant-A");

sensor.put("isOnline", "true");

sensor.put("batteryLevel", "85");

sensor.put("firmwareVersion", "2.4.1");
```

A client should be able to build a filter representing the following criteria:

- The asset operates in the zone `"Lombardy-Plant-A"`.
- The asset is currently online (`isOnline` is `"true"`).
- The asset has a battery level of at least `20`%.

The resulting filter should match the asset above.

If the sensor's battery level drops to `10`%, the same filter should no longer match.

The example illustrates the intended usage, not a required API design. You are free to choose how filters and operators are represented and composed.

Constraints and assumptions

- **Language:** Java.
- **Asset representation:** `Map<String, String>`.
- **Dependencies:** Prefer the Java standard library. External libraries are allowed but should be justified by a clear benefit to the solution.

- **Testing and build tools:** Candidates are free to choose their testing framework and build tool. For example, JUnit, Maven, and Gradle are all acceptable.
- **Frameworks:** No application framework is required.
- **Execution environment:** The solution should be straightforward to compile and run.
- **Scope:** Focus exclusively on filter definition and evaluation. Assets persistence, REST APIs, scheduling, ranking, and assignment optimization are out of scope.

Some behavioural details are intentionally left open. For example, a property may be missing, or a value may not be valid for a numeric comparison.

Choose and document a consistent approach to these cases. Make sure your implementation and tests reflect the semantics you have chosen.

What we want to see

- **Clear modelling:** abstractions and responsibilities that make the API easy to understand and use.
- **Composability:** the ability to build complex filters from simpler ones without introducing unnecessary coupling.
- **Extensibility:** a design that accommodates new operators with minimal changes to existing code.
- **Correctness:** well-defined behaviour for the supported operators, including relevant edge cases.
- **Automated tests:** tests covering the main functionality, composition, and important edge cases.
- **Maintainability:** readable code, meaningful names, and a clear separation of concerns where appropriate.

Submission

Please provide:

- The Java source code, submitted either as an archive or through a Git repository (e.g. GitHub, GitLab, or a similar platform).
- Automated tests and instructions for running them.
- A short README describing how to build and run the solution.
- A brief explanation of your main design decisions, including:
    - Which operators you implemented and why.
    - How filters are represented and composed.
    - How missing properties and invalid comparison values are handled.
    - How you would extend the design with a new operator.

A complete parser, serialization mechanism, production-ready application, or extensive documentation is not expected.

Time expectation

The assignment is intended to take approximately **2–4 hours**.

Interview format

Total duration: 60 minutes

First 20 minutes — Presentation

You will have 20 minutes to walk the panel through your solution, explaining your main design decisions, the functionality you implemented, your testing approach, and any relevant trade-offs or limitations.

No slides are required. You may use your code, README, or any other format you find useful. We are interested in the substance of your engineering decisions and your ability to explain them, not in presentation polish.

Following 40 minutes — Technical discussion

The panel will explore your implementation through questions and concrete scenarios. Topics may include design and extensibility, correctness and edge cases, testing, and how your solution could evolve to meet new requirements.

The discussion will focus on your actual implementation and the reasoning behind your decisions. You may be asked to consider alternative approaches, explain trade-offs, or adapt your design to new constraints.